

**Universidad Peruana de Ciencias Aplicadas**

**Trabajo final: segundo avance (hito 2)**

**Teoría de Compiladores, sección 13962**

**Docente**: Marco Antonio Sobrevilla Cabezudo

Grupo 5

**Tema:** PixelLang, un lenguaje para describir la máquina de estados de los personajes de un videojuego 2D

Integrantes:



| Código | Apellidos y nombres |
|---|---|
| U20221G222 | Ruiz Soto, Diego Gilmer |
| U202417368 | Ramos Parihuaman, David Miguel |
| U20241D937 | Quito Anccasi, Antony Rodrigo |
| U202412516 | Monge Jiménez, Mateo Alonso |



Noviembre de 2026

# PixelLang: un lenguaje para describir la máquina de estados de los personajes de un videojuego 2D

## Problemática y motivación

En un videojuego 2D, el comportamiento de cada personaje se describe con una máquina de estados finitos. El personaje está en reposo, saltando, atacando o aturdido, y cambia de estado cuando ocurre un evento, como pulsar un botón o recibir un golpe (Nystrom, 2014). Sin embargo, esa máquina casi nunca se escribe como tal. En el código del juego queda repartida en condicionales que revisan banderas como *saltando* o *atacando*, y nada impide que dos banderas incompatibles queden activas a la vez (Nystrom, 2014). En los editores visuales de motores como Unity, los estados se dibujan como nodos, pero las condiciones de cada transición se configuran aparte, en el panel Inspector (Unity Technologies, s. f.-a).

Este problema es importante porque la calidad de un juego es esencial para su éxito y, aun así, los videojuegos suelen publicarse con errores desde el primer día, cualquiera que sea su presupuesto o el tamaño del equipo (Politowski et al., 2021). La máquina de estados decide qué puede hacer el personaje en cada momento, por lo que un error en ella llega directamente al jugador. Un estado al que nunca se llega es una acción que no se puede usar, una transición hacia un estado que ya no existe deja al personaje sin respuesta y un estado del que no se puede salir lo deja atrapado, lo que puede impedir que la partida continúe. El compilador del lenguaje en que está escrito el juego no detecta ninguno de estos errores, porque para él las banderas son variables como cualquier otra.

Actualmente, estos errores solo se detectan jugando, y es un proceso largo. Para revisar la máquina hay que llevar al personaje a cada estado y disparar cada evento: un personaje sencillo, con 5 estados y 6 eventos, ya tiene 30 combinaciones de estado y evento, y cada estado nuevo agrega 6 más. Esa revisión se hace a mano y se repite cada vez que cambia el diseño, porque los desarrolladores dependen casi exclusivamente de las partidas de prueba manuales (Politowski et al., 2021) y evitan las pruebas automatizadas, porque quedan obsoletas con cada cambio del diseño (Murphy-Hill et al., 2014). A esto se suma que los diagramas de estados planos se vuelven difíciles de mantener a medida que crecen (Harel, 1987). Existen formatos como SCXML (World Wide Web Consortium, 2015) y bibliotecas como XState (Stately, s. f.) que describen máquinas de estados en texto, pero son de propósito general y están pensados para integrarse en intérpretes o en aplicaciones web, no para describir los personajes de un juego.

Por eso proponemos PixelLang, un lenguaje de dominio específico en el que la máquina de estados de cada personaje se escribe como texto. Un lenguaje de este tipo permite analizar y verificar los programas en los términos del dominio (Mernik et al., 2005). En PixelLang, cada personaje es un autómata finito $A = \{Q, \Sigma, \delta, q_0, F\}$, de modo que el compilador puede comprobar, antes de ejecutar nada, que ese autómata esté bien formado. Así, los errores que hoy aparecen durante una partida se detectan al compilar, con su línea y su columna, sin necesidad de escribir pruebas para ellos. Nos motiva, además, que el problema permita aplicar directamente los temas del curso: autómatas finitos, gramáticas libres de contexto y las fases de un compilador.

Lo que diferencia a PixelLang de esas alternativas es que su compilador conoce el dominio: sabe qué es un estado, un evento y una transición, y por eso rechaza al compilar errores que con ellas se descubren recién al ejecutar el juego. Además, el compilador genera el código del personaje. En el hito 3, ese código se compilará con LLVM como una biblioteca nativa que un motor como Unity puede llamar desde sus scripts de C# (Unity Technologies, s. f.-b): el juego solo enviará los eventos y consultará el estado, sin volver a programar la máquina.

## Objetivos

El objetivo general es desarrollar un compilador para PixelLang que verifique, al compilar, que la máquina de estados de cada personaje esté bien formada, y que la traduzca a código ejecutable. Los objetivos específicos son los siguientes:

1. Definir el léxico del lenguaje e implementar el analizador léxico con expresiones regulares en ANTLR4.
2. Diseñar una gramática libre de contexto para cada construcción, validarla con derivaciones más a la izquierda e implementar el analizador sintáctico en ANTLR4.
3. Construir la tabla de símbolos e implementar un analizador semántico que detecte los errores del autómata.
4. Construir el árbol de sintaxis abstracta de cada programa y traducirlo a código intermedio de tres direcciones, representado también como cuádruplos y triples.
5. Generar código ejecutable con LLVM (Lattner y Adve, 2004) y empaquetarlo como una biblioteca nativa que se pueda usar desde Unity.

## Construcciones del lenguaje

Un programa de PixelLang declara uno o más personajes y, al final, cero o más simulaciones. Cada personaje declara sus atributos, sus eventos y sus estados, y cada estado lista las transiciones que salen de él. La Tabla 1 resume las siete construcciones del lenguaje con un ejemplo de cada una, y el Apéndice A contiene los cinco ejemplos de cada construcción.

**Tabla 1**

*Construcciones de PixelLang*

| Construcción | Descripción | Ejemplo |
|---|---|---|
| Programa y declaración de personaje | Lista de personajes seguida de simulaciones. Un personaje contiene sus atributos, sus eventos y al menos un estado. | personaje Puerta {<br>  eventos ser_abierta;<br>  inicial estado Cerrada {<br>  cuando ser_abierta -> Abierta;<br>  }<br>  final estado Abierta { }<br>}<br>simular Puerta: ser_abierta; |
| Declaración de atributos | Variable del personaje con nombre, tipo y valor inicial. | vida: entero = 100; |
| Declaración de eventos | Lista de los eventos del personaje, que forman su alfabeto. | eventos saltar, atacar; |
| Declaración de estados | Estado con un modificador opcional (*inicial* o *final*), acciones al entrar y transiciones. | estado Saltando {<br>  al_entrar { y = y + 2.5; }<br>  cuando aterrizar -> Reposo;<br>  cuando recibir_golpe -> Aturdido;<br>} |
| Transiciones | Evento que la dispara, guarda opcional con *si* y estado destino. | cuando fin_aturdimiento si vida <= 0<br>  -> Derrotado; |
| Acciones: asignación y selección | Asignación a un atributo y selección *si* con *sino* opcional. | si vida < 50 { velocidad = 1.0; } |

| Construcción | Descripción | Ejemplo |
|---|---|---|
| Expresiones | Operaciones lógicas, relacionales y aritméticas con precedencia. | municion > 0 && escudo != verdadero |

## Analizador léxico

La Tabla 2 presenta el léxico de PixelLang en la forma token: lista de lexemas. El analizador léxico está implementado con expresiones regulares en ANTLR4 (Parr, 2013). Siempre gana el lexema más largo, por lo que *siguiente* es un identificador y no la palabra *si* seguida de otras letras. Si dos reglas reconocen un lexema de la misma longitud, gana la que aparece primero; por eso las palabras reservadas se definen antes que ID.

**Tabla 2**

*Léxico de PixelLang en la forma token: lista de lexemas*

| Categoría | Token: lista de lexemas |
|---|---|
| Palabras reservadas | PERSONAJE: **personaje** EVENTOS: **eventos** ESTADO: **estado**<br>INICIAL: **inicial** FINAL: **final** AL_ENTRAR: **al_entrar**<br>CUANDO: **cuando** SI: **si** SINO: **sino** SIMULAR: **simular** |
| Tipos y valores lógicos | ENTERO: **entero** REAL: **real** LOGICO: **logico**, **lógico**<br>VERDADERO: **verdadero** FALSO: **falso** |
| Operadores | FLECHA: -> O_LOGICO: \|\| Y_LOGICO: **&&** NO_LOGICO: **!**<br>IGUAL: == DISTINTO: **!=** MENOR_IGUAL: <=<br>MAYOR_IGUAL: >= MENOR: < MAYOR: > ASIGNA: =<br>MAS: + MENOS: - POR: * ENTRE: / |
| Signos de puntuación | PAR_A: **(** PAR_C: **)** LLAVE_A: **{** LLAVE_C: **}**<br>DOS_PUNTOS: **:** PUNTO_COMA: **;** COMA: **,** |
| Números e identificadores | NUM_ENTERO: **[0-9]+** (por ejemplo, 0, 25, 100)<br>NUM_REAL: **[0-9]+ '.' [0-9]+** (por ejemplo, 0.0, 2.5, 9.8)<br>ID: **LETRA (LETRA \| DIGITO)*** (por ejemplo, vida, y, Caballero, daño_golpe) |
| Se descartan | COMENTARIO_LINEA: **'//' ~[\r\n]***<br>COMENTARIO_BLOQUE: **'/*' .*? '*/'**<br>ESPACIOS: **[ \t\r\n]+** |

*Nota.* Los lexemas van en negrita. LETRA incluye las vocales con tilde, la ü, la ñ y el guion bajo; DIGITO es un dígito del 0 al 9.

## Analizador sintáctico

La Tabla 3 contiene las gramáticas libres de contexto de las siete construcciones, escritas con la notación vista en clase (Aho et al., 2006): los no terminales van en cursiva, los terminales en negrita, las alternativas se separan con | y ε es la cadena vacía. Los terminales **id**, **num** y **dec** representan los tokens ID, NUM_ENTERO y NUM_REAL. La gramática de expresiones tiene un no terminal por cada nivel de precedencia, por lo que no es ambigua.

**Tabla 3**
*Gramáticas libres de contexto de PixelLang*

| Gramáticas G1 a G4 | Gramáticas G5 a G7 |
|---|---|
| **G1. Programa y declaración de personaje**<br>*programa* → *lista_pers lista_sims*<br>*lista_pers* → *lista_pers decl_pers* \| *decl_pers*<br>*lista_sims* → *lista_sims simulacion* \| ε<br>*decl_pers* → **personaje id {** *lista_atrs decl_evs lista_est* **}**<br>*lista_atrs* → *lista_atrs decl_atr* \| ε<br>*lista_est* → *lista_est decl_est* \| *decl_est*<br>*simulacion* → **simular id :** *lista_ids* **;**<br>**G2. Declaración de atributos**<br>*decl_atr* → **id :** *tipo* **=** *expr* **;**<br>*tipo* → **entero** \| **real** \| **logico**<br>**G3. Declaración de eventos**<br>*decl_evs* → **eventos** *lista_ids* **;**<br>*lista_ids* → *lista_ids* **, id** \| **id**<br>**G4. Declaración de estados**<br>*decl_est* → *modif* **estado id {** *entrada lista_trs* **}**<br>*modif* → **inicial** \| **final** \| ε<br>*entrada* → **al_entrar** *bloque* \| ε<br>*lista_trs* → *lista_trs transicion* \| ε | **G5. Transiciones**<br>*transicion* → **cuando id** *guarda* **-> id ;**<br>*guarda* → **si** *expr* \| ε<br>**G6. Acciones: asignación y selección**<br>*sentencia* → **id =** *expr* **;** \| **si** *expr bloque alternativa*<br>*alternativa* → **sino** *bloque* \| ε<br>*bloque* → **{** *lista_sent* **}**<br>*lista_sent* → *lista_sent sentencia* \| ε<br>**G7. Expresiones**<br>*expr* → *expr* **\|\|** *conj* \| *conj*<br>*conj* → *conj* **&&** *rel* \| *rel*<br>*rel* → *arit oprel arit* \| *arit*<br>*oprel* → **<** \| **>** \| **<=** \| **>=** \| **==** \| **!=**<br>*arit* → *arit* **+** *term* \| *arit* **-** *term* \| *term*<br>*term* → *term* ***** *unario* \| *term* **/** *unario* \| *unario*<br>*unario* → **!** *unario* \| **-** *unario* \| *factor*<br>*factor* → **(** *expr* **)** \| **num** \| **dec** \| **verdadero** \| **falso** \| **id** |

Para validar cada gramática derivamos por la izquierda cuatro de sus ejemplos. Las 28 derivaciones están en el Apéndice B y la Tabla 4 muestra una de ellas. Cuando el no terminal más a la izquierda pertenece a otra gramática, su subderivación se resume en un solo paso con ⇒*.

**Tabla 4**
*Derivación más a la izquierda del ejemplo 5.2*

| Paso | Forma sentencial | Producción aplicada |
|---|---|---|
| 0 | *transicion* | *transicion* → **cuando id** *guarda* **-> id ;** |
| 1 | **cuando id** *guarda* **-> id ;** | *guarda* → **si** *expr* |
| 2 | **cuando id si** *expr* **-> id ;** | *expr* ⇒* **id <= num** (G7) |
| 3 | **cuando id si id <= num -> id ;** | *fin* |

*Nota.* El ejemplo 5.2 es la transición “cuando fin_aturdimiento si vida <= 0 -> Derrotado;”. El paso 2 resume la subderivación de *expr* en la gramática G7.

## Gramática actualizada

Para el hito 2 actualizamos la gramática de ANTLR4 (gramatica/PixelLang.g4) con el fin de construir el árbol de sintaxis abstracta (AST). Como en los laboratorios 6 y 7, cada alternativa de las reglas *unario* y *factor* tiene ahora una etiqueta, de modo que ANTLR4 genera una clase de contexto y un método del *visitor* por cada una. Así, cada método construye directamente el nodo del AST que le corresponde, como muestra la Tabla 5.

**Tabla 5**

*Alternativas etiquetadas de las reglas unario y factor*

| Regla | Alternativa | Etiqueta | Nodo del AST |
|---|---|---|---|
| unario | NO_LOGICO unario | negacion | Unaria("!", operando) |
| unario | MENOS unario | negativo | Unaria("-", operando) |
| unario | factor | primario | el nodo del factor |
| factor | PAR_A expr PAR_C | parentesis | el nodo de la expresión; los paréntesis no crean nodo |
| factor | NUM_ENTERO | numEntero | Numero(valor entero) |
| factor | NUM_REAL | numReal | Numero(valor real) |
| factor | (VERDADERO \| FALSO) | valorLogico | Logico(valor) |
| factor | ID | variable | Variable(nombre) |

*Nota.* En el hito 1, *unario* tenía una sola alternativa para los dos operadores, (NO_LOGICO | MENOS) unario; ahora tiene una por operador, y el nodo Unaria guarda cuál es. Las palabras verdadero y falso se agruparon en una sola alternativa.

Las etiquetas no cambian el lenguaje: la gramática acepta las mismas cadenas, por lo que las gramáticas libres de contexto de la Tabla 3 y las derivaciones del Apéndice B siguen siendo válidas. El analizador semántico cambió de forma, pero no de reglas: sus métodos visitUnario y visitFactor se dividieron en un método por etiqueta. Para comprobarlo, volvimos a ejecutar las pruebas del hito 1 y los 35 ejemplos de las construcciones, y la salida del compilador fue idéntica a la anterior.

## Analizador semántico

El analizador semántico es un *visitor* de ANTLR4 que recorre el árbol sintáctico y construye la tabla de símbolos. La tabla tiene dos niveles de ámbito: el global, con los personajes, y uno por cada personaje, con sus atributos y tipos, sus eventos y sus estados. Con esa tabla verifica los diez errores de la Tabla 6: seis corresponden a la definición de autómata finito (E01 a E05 y E09) y cuatro, a los nombres, los tipos y las simulaciones (E06, E07, E08 y E10). La función de transición δ solo puede llevar a estados de Q, y para cada estado y evento se toma un único destino: las transiciones con el mismo evento se revisan en orden y se toma la primera cuya guarda se cumple, por lo que una transición sin guarda debe ser la última con ese evento.

**Tabla 6**

*Errores semánticos que detecta el compilador*

| Código | Error | Regla que viola | Ejemplo |
|---|---|---|---|
| E01 | Transición hacia un estado no declarado | δ solo puede llevar a estados de Q | estado Saltando {<br>cuando saltar -> Volando;<br>} |
| E02 | Evento no declarado, en una transición o en una simulación | Los símbolos de entrada pertenecen a Σ | estado Saltando { cuando caer -><br>Reposo; } |

| Código | Error | Regla que viola | Ejemplo |
|---|---|---|---|
| E03 | Transiciones no deterministas | δ(q, e) tiene un único destino | cuando fin_aturdimiento -> Reposo;<br>cuando fin_aturdimiento si vida <= 0 -> Derrotado; |
| E04 | Estado inicial ausente o repetido | Existe exactamente un $q_0$ | estado Reposo { cuando saltar -> Saltando; }<br>estado Saltando { cuando aterrizar -> Reposo; } |
| E05 | Estado inalcanzable desde el estado inicial | Todo estado se alcanza desde $q_0$ | estado Nadando { cuando aterrizar -> Reposo; } |
| E06 | Nombre repetido o usado para dos cosas | Cada nombre identifica una sola cosa | vida: entero = 100;<br>vida: real = 50.0; |
| E07 | Atributo no declarado | Todo atributo usado está en la tabla de símbolos | al_entrar { escudo = 10; } |
| E08 | Tipos incompatibles en expresiones, guardas y asignaciones | Reglas de tipos del lenguaje | vida: entero = 2.5;<br>si vida { vivo = falso; } |
| E09 | Estado final con transiciones de salida | Un estado de F no tiene salidas | final estado Derrotado {<br>cuando revivir -> Reposo;<br>} |
| E10 | Simulación de un personaje no declarado | La simulación usa un personaje existente | simular Mago: recibir_golpe; |

*Nota.* Cada ejemplo es un fragmento de la prueba de ese error (carpeta pruebas/semantico). En E01, Volando no es un estado del personaje; en E02, caer no está en su lista de eventos; en E03, la transición con guarda nunca se toma, porque la anterior responde al mismo evento sin guarda; en E04, ninguno de los dos estados es inicial; en E05, ninguna transición llega a Nadando; en E07, escudo no es un atributo; en E08, un atributo entero recibe un real y un entero se usa como condición; y en E10, no existe el personaje Mago.

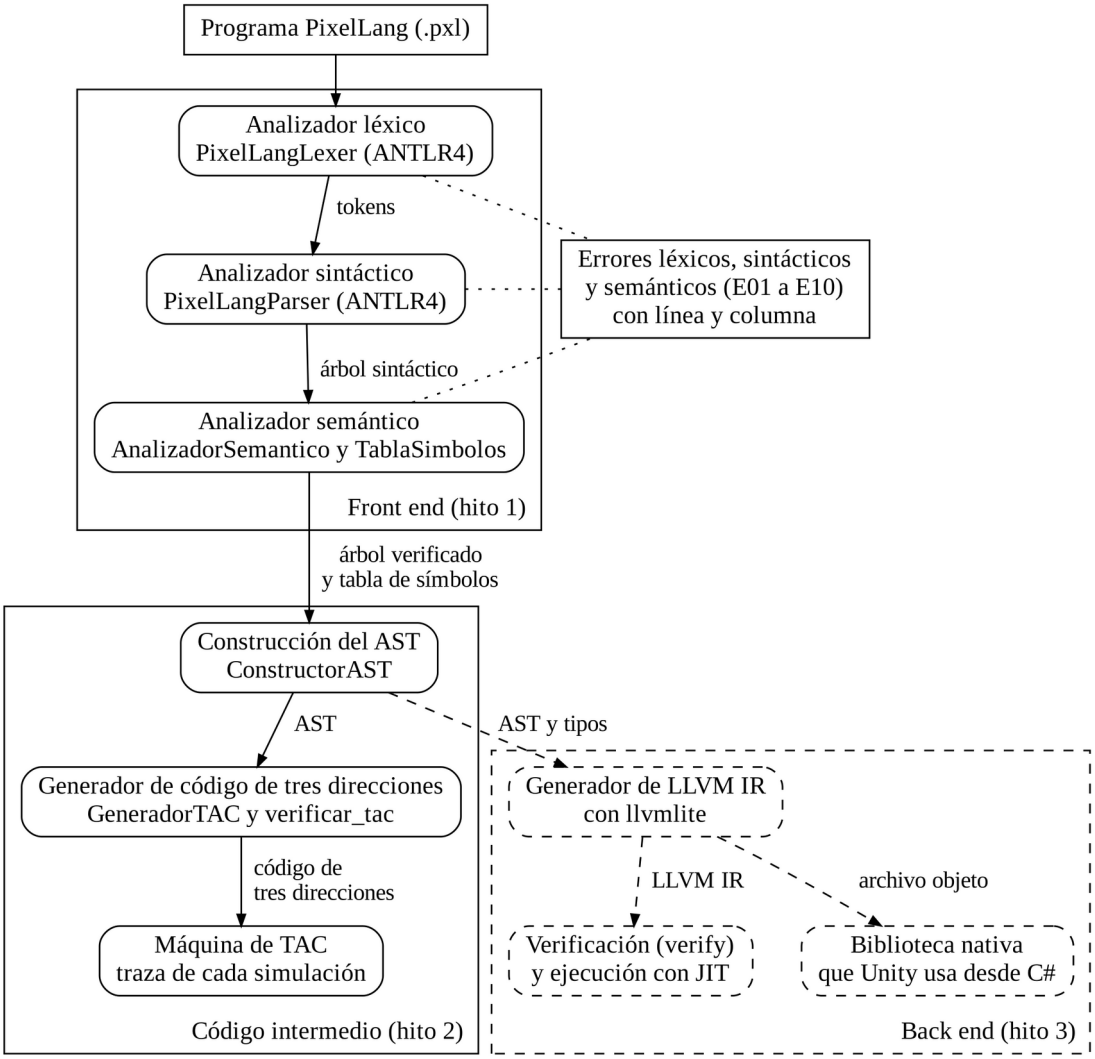
Además, el analizador advierte cuando un estado que no es final no tiene transiciones de salida, porque el personaje quedaría atrapado en él; no es un error, porque puede ser intencional mientras se diseña. Cada error se informa con su código, su línea y su columna (contada desde 0, como en ANTLR), y el análisis continúa para reportar todos los errores juntos. Si hay algún error, el compilador no construye el AST ni genera código. Las pruebas de los tres analizadores, con entradas válidas e inválidas, están en el Apéndice C.

## Arquitectura del compilador

La Figura 1 muestra las fases del compilador de PixelLang y lo que pasa de una a otra. Siguen el esquema del proceso de compilación visto en clase (Aho et al., 2006): el *front end* analiza el programa y construye la tabla de símbolos; con el AST, los tipos y los símbolos, el generador produce el código intermedio; y el *back end* lo traduce a LLVM IR y a código de máquina. Las fases del *front end* y la generación de código intermedio ya están implementadas; el *back end* corresponde al hito 3.

**Figura 1**

*Arquitectura del compilador de PixelLang*




*Nota.* Las cajas con borde discontinuo corresponden al hito 3. Cada fase del *front end* detiene la compilación si encuentra errores, y el compilador los informa todos juntos.

Cada fase está en un módulo de Python que el driver (main.py) llama en orden. La Tabla 7 resume, para cada módulo, lo que recibe, lo que produce y la opción del driver que muestra su resultado.

**Tabla 7**

*Módulos del compilador*

| Fase | Módulo | Entrada | Salida | Opción |
|---|---|---|---|---|
| Análisis léxico | PixelLangLexer, generado por ANTLR4 | Texto del programa | Tokens y errores léxicos | --tokens |
| Análisis sintáctico | PixelLangParser, generado por ANTLR4; frontend.py reúne los errores | Tokens | Árbol sintáctico y errores sintácticos | --arbol |

| Fase | Módulo | Entrada | Salida | Opción |
|---|---|---|---|---|
| Análisis semántico | semantico.py | Árbol sintáctico | Tabla de símbolos, errores E01 a E10 y advertencias | --tabla |
| Construcción del AST | ast_pixellang.py | Árbol sintáctico sin errores | AST | --ast |
| Generación de código intermedio | codigo_intermedio.py | AST y tabla de símbolos | Código de tres direcciones, cuádruplos y triples | --tac --cuadruplos --triples |
| Ejecución del código intermedio | maquina_tac.py | Código de tres direcciones | Traza de cada simulación | --simular |
| Generación de código (hito 3) | Por implementar, con llvmlite | AST y tabla de símbolos | LLVM IR, ejecución con JIT y biblioteca nativa | Pendiente |

*Nota.* Los módulos están en la carpeta src del repositorio. El analizador léxico y el sintáctico se generan con ANTLR4 a partir de la gramática, en la carpeta src/generado. Sin opciones, el driver solo indica si el programa tiene errores y cuáles son.

## Generación de código intermedio

El generador recorre el AST en postorden, como en el laboratorio 6: cada operación se emite como una instrucción de tres direcciones y su resultado queda en un temporal nuevo (t1, t2, etc.). Los tipos salen de la tabla de símbolos, y cuando un entero se combina con un real se inserta la conversión intToFloat, según la regla de promoción numérica de la semana 10. Las condiciones de las guardas y de las sentencias si se traducen a saltos con cortocircuito, con el algoritmo genCond (Aho et al., 2006): en A && B, B solo se evalúa si A es verdadera, y en A || B, solo si A es falsa.

Cada personaje P se traduce en tres tipos de procedimientos. P.iniciar da su valor inicial a cada atributo y entra al estado inicial. Por cada estado E hay un procedimiento P.al_entrar_E, que guarda E en la variable estado y ejecuta las acciones de al_entrar. Por cada evento ev hay un procedimiento P.cuando_ev, que implementa la función de transición δ(q, ev): compara el estado actual con cada estado que responde a ev y, en el orden en que se escribieron, toma la primera transición cuya guarda se cumple; si ninguna se cumple, el personaje se queda en su estado. Cada bloque simular se traduce en un procedimiento que llama a P.iniciar y luego al procedimiento de cada evento. La Tabla 8 muestra dos fragmentos del caballero y su traducción, y el código completo está en el Apéndice D.

**Tabla 8**

*Traducción de dos fragmentos del caballero a código de tres direcciones*

| **PixelLang** | **Código de tres direcciones** |
|---|---|
| estado Atacando {<br>  al_entrar {<br>  si vida < 50 { velocidad = 1.0; }<br>  sino { velocidad = 1.5; }<br>  }<br>  cuando fin_ataque -> Reposo;<br>  cuando recibir_golpe -> Aturdido;<br>} | Caballero.al_entrar_Atacando:<br>  estado = Atacando<br>  t2 = vida < 50<br>  if t2 goto L1<br>  goto L2<br>L1:<br>  velocidad = 1.0<br>  goto L3<br>L2:<br>  velocidad = 1.5<br>L3:<br>  return |
| estado Aturdido {<br>  al_entrar { ... }<br>  cuando recibir_golpe -> Aturdido;<br>  cuando fin_aturdimiento si vida <= 0 -><br>  Derrotado;<br>  cuando fin_aturdimiento -> Reposo;<br>} | Caballero.cuando_fin_aturdimiento:<br>  t12 = estado == Aturdido<br>  ifFalse t12 goto L12<br>  t13 = vida <= 0<br>  if t13 goto L13<br>  goto L14<br>L13:<br>  call Caballero.al_entrar_Derrotado, 0<br>  return<br>L14:<br>  call Caballero.al_entrar_Reposo, 0<br>  return<br>L12:<br>  return |

*Nota.* En el primer fragmento, la condición del si salta a L1 (entonces) o a L2 (sino). En el segundo, el procedimiento solo hace algo si el estado actual es Aturdido; la guarda vida <= 0 se prueba primero, y la transición sin guarda queda como la última opción.

El mismo código se puede ver como cuádruplos (operador, argumento 1, argumento 2 y resultado) o como triples, en los que cada temporal se reemplaza por la posición de la instrucción que lo calcula, como en el laboratorio 6. Antes de usarlo, la función verificar_tac revisa las condiciones de corrección de la traducción que no requieren ejecutar el código: cada temporal se define antes de usarse, los destinos de los saltos y de las llamadas existen y cada procedimiento termina con return. Por último, una máquina de TAC ejecuta el código intermedio de cada bloque simular y muestra, después de cada evento, el estado del personaje y sus atributos. Los enteros son de 32 bits, como el tipo i32 de LLVM. Si un resultado entero se sale de ese rango, o si se divide entre cero, la simulación se detiene con un error de ejecución.

## Generación de código con LLVM y conexión con Unity (hito 3)

En el hito 3, un generador recorrerá el AST con llvmlite, la biblioteca del laboratorio 7, y producirá LLVM IR (Lattner y Adve, 2004). El código de LLVM seguirá el mismo esquema que el código intermedio: cada procedimiento será una función y cada etiqueta, un bloque básico; los atributos se guardarán en variables globales, los enteros serán de tipo i32 y los reales, de tipo double. El módulo se verificará con verify() y las simulaciones se ejecutarán con el compilador JIT.

Para conectar PixelLang con Unity, llvmlite puede emitir el módulo como un archivo objeto para el enlazador de la plataforma (llvmlite, s. f.), que se enlaza como una biblioteca nativa (un archivo .dll en Windows). Un script de C# declara sus funciones con el atributo DllImport y las llama como a cualquier otro método (Unity Technologies, s. f.-b). Así, cuando el jugador pulsa una tecla, el script llama a la función del evento y luego consulta el estado para elegir la animación, como en el esbozo de la Tabla 9. La lógica del personaje queda escrita y verificada en PixelLang, y Unity solo la usa.

**Tabla 9**

*Uso previsto del caballero compilado desde un script de Unity*

**Script de C# (esbozo para el hito 3)**

```
using System.Runtime.InteropServices;
using UnityEngine;

public class Caballero : MonoBehaviour {
    [DllImport("caballero")] static extern void Caballero_iniciar();
    [DllImport("caballero")] static extern void Caballero_cuando_saltar();
    [DllImport("caballero")] static extern int Caballero_estado();
    string[] estados = { "Reposo", "Saltando", "Atacando", "Aturdido", "Derrotado" };
    Animator animator;
    int anterior = -1;

    void Start() {
        animator = GetComponent<Animator>();
        Caballero_iniciar();
    }

    void Update() {
        if (Input.GetKeyDown(KeyCode.Space)) Caballero_cuando_saltar();
        int estado = Caballero_estado();
        if (estado != anterior) { animator.Play(estados[estado]); anterior = estado; }
    }
}
```

*Nota.* Las funciones Caballero_iniciar, Caballero_cuando_saltar y Caballero_estado serían las que exporte la biblioteca generada por PixelLang; el estado se devuelve como un número, en el orden en que se declararon los estados.

## Plan de validación del compilador

La validación revisa cada fase con pruebas automáticas que están en la carpeta pruebas del repositorio; cada una se ejecuta con un solo comando, como indica la guía (GUIA.md). La Tabla 10 resume el plan: qué se prueba en cada nivel, el criterio para aceptarlo y el hito en que se completa. Verificar que el código esté bien formado no demuestra que conserve el significado del programa; por eso, además de comparar el código generado con el esperado, el plan ejecuta ese código y compara sus resultados.

**Tabla 10**

*Plan de validación del compilador*

| Nivel | Pruebas | Criterio de aceptación | Hito |
|---|---|---|---|
| 1. Léxico | 6 entradas de pruebas/lexico, con pruebas/correr_pruebas.py | La entrada válida produce los tokens esperados y cada entrada inválida, su error con línea y columna | 1 |
| 2. Sintáctico | Los 35 ejemplos de las construcciones (probar_construcciones.py) y 6 programas con errores (pruebas/sintactico) | Los 35 ejemplos se aceptan y cada programa inválido informa el error en la línea donde está | 1 |
| 3. Semántico | 10 programas, uno por error (pruebas/semantico), y los dos ejemplos del repositorio | Cada programa informa su error con código, línea y columna; los ejemplos no tienen errores | 1 |
| 4. AST | probar_ast.py: precedencia, asociatividad, operadores unarios y estructura del caballero | Se cumplen todas las aserciones | 2 |
| 5. Código intermedio | probar_tac.py: TAC de expresiones (laboratorio 6), conversiones, cortocircuito y si/sino (semana 10), cuádruplos y triples | El TAC coincide con el esperado y verificar_tac no encuentra problemas | 2 |
| 6. Ejecución del código intermedio | Trazas del caballero, el slime, una puerta, una división entre cero y un desbordamiento de entero (probar_tac.py); prueba diferencial con programas al azar (probar_generados.py) | Las trazas son las esperadas y coinciden con las del intérprete de referencia en todos los programas | 2 |
| 7. Regresión | Todas las pruebas anteriores, antes de cada commit | Las salidas de los hitos anteriores no cambian | 2 y 3 |
| 8. Código LLVM | verify() del módulo y ejecución con JIT de las simulaciones; los mismos programas al azar | El IR pasa la verificación y cada simulación da la misma traza que la máquina de TAC | 3 |
| 9. Integración con Unity | Escena de prueba con un script de C# que llama a la biblioteca del caballero | El estado que lee Unity coincide con la traza de la simulación | 3 |

La prueba diferencial del nivel 6 compara dos implementaciones independientes de la semántica de PixelLang. El generador de pruebas (pruebas/generador.py) crea programas válidos al azar, con todos los operadores, enteros combinados con reales, guardas con && y ||, sentencias si anidadas y divisiones que pueden ser entre cero. Cada programa se traduce a código de tres direcciones y sus simulaciones se ejecutan dos veces: en la máquina de TAC y en un intérprete de referencia que recorre el AST sin pasar por el código intermedio (pruebas/referencia.py). Si las dos trazas difieren en algún evento, hay un error en la traducción o en la máquina.

Al cerrar el hito 2 se cumplen todas las pruebas de los niveles 1 a 6. En la prueba diferencial, 3000 programas generados con la semilla 7 produjeron 6779 simulaciones y 40 210 eventos, y las dos implementaciones coincidieron en todos. De esas simulaciones, 1842 terminan en un error de ejecución, casi siempre una división entre cero y, en las demás, un entero que se sale del rango de 32 bits; las dos implementaciones detectan el error en el mismo evento. En el hito 3, la misma prueba comparará la máquina de TAC con el código de LLVM.

## Referencias


Aho, A. V., Lam, M. S., Sethi, R. y Ullman, J. D. (2006). *Compilers: Principles, techniques, and tools* (2.ª ed.). Pearson.

Harel, D. (1987). Statecharts: A visual formalism for complex systems. *Science of Computer Programming, 8*(3), 231–274. https://doi.org/10.1016/0167-6423(87)90035-9

Lattner, C. y Adve, V. (2004). LLVM: A compilation framework for lifelong program analysis & transformation. En *International Symposium on Code Generation and Optimization, 2004. CGO 2004* (pp. 75–86). IEEE. https://doi.org/10.1109/CGO.2004.1281665

llvmlite. (s. f.). *Target information*. Recuperado el 5 de noviembre de 2026, de https://llvmlite.readthedocs.io/en/latest/user-guide/binding/target-information.html

Mernik, M., Heering, J. y Sloane, A. M. (2005). When and how to develop domain-specific languages. *ACM Computing Surveys, 37*(4), 316–344. https://doi.org/10.1145/1118890.1118892

Murphy-Hill, E., Zimmermann, T. y Nagappan, N. (2014). Cowboys, ankle sprains, and keepers of quality: How is video game development different from software development? En *Proceedings of the 36th International Conference on Software Engineering* (pp. 1–11). ACM. https://doi.org/10.1145/2568225.2568226

Nystrom, R. (2014). State. En *Game programming patterns*. Genever Benning. https://gameprogrammingpatterns.com/state.html

Parr, T. (2013). *The definitive ANTLR 4 reference*. The Pragmatic Bookshelf.

Politowski, C., Petrillo, F. y Guéhéneuc, Y.-G. (2021). *A survey of video game testing* [Preprint]. arXiv. https://doi.org/10.48550/arXiv.2103.06431

Stately. (s. f.). *XState*. Recuperado el 4 de octubre de 2026, de https://stately.ai/docs/xstate

Unity Technologies. (s. f.-a). *Animation transitions*. Recuperado el 6 de octubre de 2026, de https://docs.unity3d.com/Manual/class-Transition.html

Unity Technologies. (s. f.-b). *Introduction to native plug-ins in Unity*. Recuperado el 5 de noviembre de 2026, de https://docs.unity.com/en-us/engine/6000.6/manual/programming-environment/plug-ins/native/overview

World Wide Web Consortium. (2015). *State chart XML (SCXML): State machine notation for control abstraction* (W3C Recommendation). https://www.w3.org/TR/scxml/

# Anexos

**Anexo A. Repositorio del proyecto en GitHub:** https://github.com/DavidRamPari/pixellang

**Anexo B. Video de demostración del hito 1:** https://drive.google.com/file/d/1asrzQjXH87vR6JPZDanqDFVFOEntwdZ8/view?usp=sharing

**Anexo C. Guía de compilación en Visual Studio Code:** archivo GUIA.md del repositorio

# Apéndice A
# Ejemplos de las construcciones

Las Tablas A1 a A7 contienen los cinco ejemplos de cada construcción. Todos son aceptados por el analizador sintáctico a partir de la regla de su construcción, y su número se usa en las derivaciones del Apéndice B.

**Tabla A1**

*Ejemplos de la construcción programa y declaración de personaje*

| Ejemplo | Código |
|---|---|
| 1.1 | personaje Estatua {<br>  eventos ser_tocada;<br>  inicial estado Quieta {<br>  cuando ser_tocada -> Quieta;<br>  }<br>} |
| 1.2 | personaje Puerta {<br>  eventos ser_abierta;<br>  inicial estado Cerrada {<br>  cuando ser_abierta -> Abierta;<br>  }<br>  final estado Abierta { }<br>}<br>simular Puerta: ser_abierta; |
| 1.3 | personaje Moneda {<br>  valor: entero = 10;<br>  eventos ser_recogida;<br>  inicial estado Visible {<br>  cuando ser_recogida -> Recogida;<br>  }<br>  final estado Recogida { }<br>} |
| 1.4 | personaje Boton {<br>  eventos ser_pulsado;<br>  inicial estado Suelto {<br>  cuando ser_pulsado -> Suelto;<br>  }<br>}<br>personaje Bandera {<br>  eventos ser_tocada;<br>  inicial estado Abajo {<br>  cuando ser_tocada -> Arriba;<br>  }<br>  final estado Arriba { }<br>} |

| Ejemplo | Código |
|---|---|
| 1.5 | personaje Torreta {<br>  municion: entero = 3;<br>  eventos disparar, fin_disparo, fin_recarga;<br>  inicial estado Lista {<br>  cuando disparar si municion > 0 -> Disparando;<br>  cuando disparar -> Recargando;<br>  }<br>  estado Disparando {<br>  al_entrar { municion = municion - 1; }<br>  cuando fin_disparo -> Lista;<br>  }<br>  estado Recargando {<br>  al_entrar { municion = 3; }<br>  cuando fin_recarga -> Lista;<br>  }<br>}<br>simular Torreta: disparar, fin_disparo, disparar; |

**Tabla A2**

*Ejemplos de la construcción declaración de atributos*

| Ejemplo | Código |
|---|---|
| 2.1 | vida: entero = 100; |
| 2.2 | velocidad: real = 2.5; |
| 2.3 | vivo: logico = verdadero; |
| 2.4 | gravedad: real = -9.8; |
| 2.5 | daño_critico: entero = daño_golpe * 2; |

**Tabla A3**

*Ejemplos de la construcción declaración de eventos*

| Ejemplo | Código |
|---|---|
| 3.1 | eventos saltar; |
| 3.2 | eventos saltar, atacar; |
| 3.3 | eventos saltar, atacar, recibir_golpe; |
| 3.4 | eventos ver_jugador, perder_jugador, recibir_golpe; |
| 3.5 | eventos pulsar_a, pulsar_b, soltar_a, soltar_b, pausar; |

**Tabla A4**

*Ejemplos de la construcción declaración de estados*

| Ejemplo | Código |
|---|---|
| 4.1 | estado Reposo { } |
| 4.2 | inicial estado Reposo { cuando saltar -> Saltando; } |
| 4.3 | final estado Derrotado { } |

| Ejemplo | Código |
|---|---|
| 4.4 | estado Aturdido {<br>  al_entrar { vida = vida - daño_golpe; }<br>  cuando recibir_golpe -> Aturdido;<br>} |
| 4.5 | estado Saltando {<br>  al_entrar { y = y + 2.5; }<br>  cuando aterrizar -> Reposo;<br>  cuando recibir_golpe -> Aturdido;<br>} |

**Tabla A5**

*Ejemplos de la construcción transiciones*

| Ejemplo | Código |
|---|---|
| 5.1 | cuando saltar -> Saltando; |
| 5.2 | cuando fin_aturdimiento si vida <= 0 -> Derrotado; |
| 5.3 | cuando fin_ataque -> Reposo; |
| 5.4 | cuando disparar si municion > 0 && !bloqueada -> Disparando; |
| 5.5 | cuando recibir_golpe si escudo \|\| vida > 50 -> Bloqueando; |

**Tabla A6**

*Ejemplos de la construcción acciones: asignación y selección*

| Ejemplo | Código |
|---|---|
| 6.1 | vida = 100; |
| 6.2 | velocidad = velocidad * 0.5; |
| 6.3 | si vida < 50 { velocidad = 1.0; } |
| 6.4 | si vida < 50 { velocidad = 1.0; } sino { velocidad = 1.5; } |
| 6.5 | si escudo {<br>  escudo = falso;<br>} sino {<br>  vida = vida - daño_golpe;<br>  si vida <= 0 { vivo = falso; }<br>} |

**Tabla A7**

*Ejemplos de la construcción expresiones*

| Ejemplo | Código |
|---|---|
| 7.1 | vida - 25 < 10 |
| 7.2 | 2 * (velocidad + 1.5) / 4 |
| 7.3 | vida <= 0 \|\| !vivo |
| 7.4 | municion > 0 && escudo != verdadero |
| 7.5 | -gravedad * tiempo >= 9.8 |

# Apéndice B
# Derivaciones más a la izquierda

Para cada gramática derivamos cuatro de los cinco ejemplos de su construcción, elegidos para que entre los cuatro se use cada producción de la gramática. La excepción es la gramática de expresiones, con 27 producciones: quedan fuera >=, ==, el cambio de signo y **falso**, que se derivan igual que >, **!=**, la negación y **verdadero**.

En cada paso se reemplaza el no terminal más a la izquierda, y la última columna indica la producción aplicada para obtener la fila siguiente. Cuando ese no terminal pertenece a otra gramática, su subderivación se resume en un paso con ⇒*, y entre paréntesis se indica la gramática a la que pertenece. Cada derivación termina en la cadena de tokens del ejemplo.

**Tabla B1**
*Derivaciones más a la izquierda de la gramática G1 (programa y declaración de personaje)*

| Paso | Forma sentencial | Producción aplicada |
|---|---|---|
| **Ejemplo 1.1** (código en el Apéndice A) | | |
| *Cadena de tokens:* **personaje id { eventos id ; inicial estado id { cuando id -> id ; } }** | | |
| 0 | *programa* | *programa* → *lista_pers lista_sims* |
| 1 | *lista_pers lista_sims* | *lista_pers* → *decl_pers* |
| 2 | *decl_pers lista_sims* | *decl_pers* → **personaje id {** *lista_atrs decl_evs lista_est* **}** |
| 3 | **personaje id {** *lista_atrs decl_evs lista_est* **}** *lista_sims* | *lista_atrs* → ε |
| 4 | **personaje id {** *decl_evs lista_est* **}** *lista_sims* | *decl_evs* ⇒* **eventos id ;** (G3) |
| 5 | **personaje id { eventos id ;** *lista_est* **}** *lista_sims* | *lista_est* → *decl_est* |
| 6 | **personaje id { eventos id ;** *decl_est* **}** *lista_sims* | *decl_est* ⇒* **inicial estado id { cuando id -> id ; }** (G4) |
| 7 | **personaje id { eventos id ; inicial estado id { cuando id -> id ; } }** *lista_sims* | *lista_sims* → ε |
| 8 | **personaje id { eventos id ; inicial estado id { cuando id -> id ; } }** | *fin* |
| **Ejemplo 1.2** (código en el Apéndice A) | | |
| *Cadena de tokens:* **personaje id { eventos id ; inicial estado id { cuando id -> id ; } final estado id { } } simular id : id ;** | | |
| 0 | *programa* | *programa* → *lista_pers lista_sims* |
| 1 | *lista_pers lista_sims* | *lista_pers* → *decl_pers* |
| 2 | *decl_pers lista_sims* | *decl_pers* → **personaje id {** *lista_atrs decl_evs lista_est* **}** |
| 3 | **personaje id {** *lista_atrs decl_evs lista_est* **}** *lista_sims* | *lista_atrs* → ε |
| 4 | **personaje id {** *decl_evs lista_est* **}** *lista_sims* | *decl_evs* ⇒* **eventos id ;** (G3) |
| 5 | **personaje id { eventos id ;** *lista_est* **}** *lista_sims* | *lista_est* → *lista_est decl_est* |
| 6 | **personaje id { eventos id ;** *lista_est decl_est* **}** *lista_sims* | *lista_est* → *decl_est* |

| Paso | Forma sentencial | Producción aplicada |
|---|---|---|
| 7 | **personaje id { eventos id ;** *decl_est decl_est* **}** *lista_sims* | *decl_est* ⇒* **inicial estado id { cuando id -> id ; }** (G4) |
| 8 | **personaje id { eventos id ; inicial estado id { cuando id -> id ; }** *decl_est* **}** *lista_sims* | *decl_est* ⇒* **final estado id { }** (G4) |
| 9 | **personaje id { eventos id ; inicial estado id { cuando id -> id ; } final estado id { } }** *lista_sims* | *lista_sims* → *lista_sims simulacion* |
| 10 | **personaje id { eventos id ; inicial estado id { cuando id -> id ; } final estado id { } }** *lista_sims simulacion* | *lista_sims* → ε |
| 11 | **personaje id { eventos id ; inicial estado id { cuando id -> id ; } final estado id { } }** *simulacion* | *simulacion* → **simular id :** *lista_ids* **;** |
| 12 | **personaje id { eventos id ; inicial estado id { cuando id -> id ; } final estado id { } } simular id :** *lista_ids* **;** | *lista_ids* ⇒* **id** (G3) |
| 13 | **personaje id { eventos id ; inicial estado id { cuando id -> id ; } final estado id { } } simular id : id ;** | *fin* |
| **Ejemplo 1.3** (código en el Apéndice A) | | |
| *Cadena de tokens:* **personaje id { id : entero = num ; eventos id ; inicial estado id { cuando id -> id ; } final estado id { } }** | | |
| 0 | *programa* | *programa* → *lista_pers lista_sims* |
| 1 | *lista_pers lista_sims* | *lista_pers* → *decl_pers* |
| 2 | *decl_pers lista_sims* | *decl_pers* → **personaje id {** *lista_atrs decl_evs lista_est* **}** |
| 3 | **personaje id {** *lista_atrs decl_evs lista_est* **}** *lista_sims* | *lista_atrs* → *lista_atrs decl_atr* |
| 4 | **personaje id {** *lista_atrs decl_atr decl_evs lista_est* **}** *lista_sims* | *lista_atrs* → ε |
| 5 | **personaje id {** *decl_atr decl_evs lista_est* **}** *lista_sims* | *decl_atr* ⇒* **id : entero = num ;** (G2) |
| 6 | **personaje id { id : entero = num ;** *decl_evs lista_est* **}** *lista_sims* | *decl_evs* ⇒* **eventos id ;** (G3) |
| 7 | **personaje id { id : entero = num ; eventos id ;** *lista_est* **}** *lista_sims* | *lista_est* → *lista_est decl_est* |
| 8 | **personaje id { id : entero = num ; eventos id ;** *lista_est decl_est* **}** *lista_sims* | *lista_est* → *decl_est* |
| 9 | **personaje id { id : entero = num ; eventos id ;** *decl_est decl_est* **}** *lista_sims* | *decl_est* ⇒* **inicial estado id { cuando id -> id ; }** (G4) |
| 10 | **personaje id { id : entero = num ; eventos id ; inicial estado id { cuando id -> id ; }** *decl_est* **}** *lista_sims* | *decl_est* ⇒* **final estado id { }** (G4) |
| 11 | **personaje id { id : entero = num ; eventos id ; inicial estado id { cuando id -> id ; } final estado id { } }** *lista_sims* | *lista_sims* → ε |
| 12 | **personaje id { id : entero = num ; eventos id ; inicial estado id { cuando id -> id ; } final estado id { } }** | *fin* |
| **Ejemplo 1.4** (código en el Apéndice A) | | |
| *Cadena de tokens:* **personaje id { eventos id ; inicial estado id { cuando id -> id ; } } personaje id { eventos id ; inicial estado id { cuando id -> id ; } final estado id { } }** | | |

| Paso | Forma sentencial | Producción aplicada |
|---|---|---|
| 0 | *programa* | *programa* → *lista_pers lista_sims* |
| 1 | *lista_pers lista_sims* | *lista_pers* → *lista_pers decl_pers* |
| 2 | *lista_pers decl_pers lista_sims* | *lista_pers* → *decl_pers* |
| 3 | *decl_pers decl_pers lista_sims* | *decl_pers* → **personaje id {** *lista_atrs decl_evs lista_est* **}** |
| 4 | **personaje id {** *lista_atrs decl_evs lista_est* **}** *decl_pers lista_sims* | *lista_atrs* → ε |
| 5 | **personaje id {** *decl_evs lista_est* **}** *decl_pers lista_sims* | *decl_evs* ⇒* **eventos id ;** (G3) |
| 6 | **personaje id { eventos id ;** *lista_est* **}** *decl_pers lista_sims* | *lista_est* → *decl_est* |
| 7 | **personaje id { eventos id ;** *decl_est* **}** *decl_pers lista_sims* | *decl_est* ⇒* **inicial estado id { cuando id -> id ; }** (G4) |
| 8 | **personaje id { eventos id ; inicial estado id { cuando id -> id ; } }** *decl_pers lista_sims* | *decl_pers* → **personaje id {** *lista_atrs decl_evs lista_est* **}** |
| 9 | **personaje id { eventos id ; inicial estado id { cuando id -> id ; } } personaje id {** *lista_atrs decl_evs lista_est* **}** *lista_sims* | *lista_atrs* → ε |
| 10 | **personaje id { eventos id ; inicial estado id { cuando id -> id ; } } personaje id {** *decl_evs lista_est* **}** *lista_sims* | *decl_evs* ⇒* **eventos id ;** (G3) |
| 11 | **personaje id { eventos id ; inicial estado id { cuando id -> id ; } } personaje id { eventos id ;** *lista_est* **}** *lista_sims* | *lista_est* → *lista_est decl_est* |
| 12 | **personaje id { eventos id ; inicial estado id { cuando id -> id ; } } personaje id { eventos id ;** *lista_est decl_est* **}** *lista_sims* | *lista_est* → *decl_est* |
| 13 | **personaje id { eventos id ; inicial estado id { cuando id -> id ; } } personaje id { eventos id ;** *decl_est decl_est* **}** *lista_sims* | *decl_est* ⇒* **inicial estado id { cuando id -> id ; }** (G4) |
| 14 | **personaje id { eventos id ; inicial estado id { cuando id -> id ; } } personaje id { eventos id ; inicial estado id { cuando id -> id ; }** *decl_est* **}** *lista_sims* | *decl_est* ⇒* **final estado id { }** (G4) |
| 15 | **personaje id { eventos id ; inicial estado id { cuando id -> id ; } } personaje id { eventos id ; inicial estado id { cuando id -> id ; } final estado id { } }** *lista_sims* | *lista_sims* → ε |
| 16 | **personaje id { eventos id ; inicial estado id { cuando id -> id ; } } personaje id { eventos id ; inicial estado id { cuando id -> id ; } final estado id { } }** | *fin* |

**Tabla B2**
*Derivaciones más a la izquierda de la gramática G2 (declaración de atributos)*

| Paso | Forma sentencial | Producción aplicada |
|---|---|---|
| **Ejemplo 2.1**: vida: entero = 100; | | |
| *Cadena de tokens:* **id : entero = num ;** | | |
| 0 | *decl_atr* | *decl_atr* → **id :** *tipo* **=** *expr* **;** |

| Paso | Forma sentencial | Producción aplicada |
|---|---|---|
| 1 | **id :** *tipo* **=** *expr* **;** | *tipo* → **entero** |
| 2 | **id : entero =** *expr* **;** | *expr* ⇒* **num** (G7) |
| 3 | **id : entero = num ;** | *fin* |
| **Ejemplo 2.2**: velocidad: real = 2.5; | | |
| *Cadena de tokens:* **id : real = dec ;** | | |
| 0 | *decl_atr* | *decl_atr* → **id :** *tipo* **=** *expr* **;** |
| 1 | **id :** *tipo* **=** *expr* **;** | *tipo* → **real** |
| 2 | **id : real =** *expr* **;** | *expr* ⇒* **dec** (G7) |
| 3 | **id : real = dec ;** | *fin* |
| **Ejemplo 2.3**: vivo: logico = verdadero; | | |
| *Cadena de tokens:* **id : logico = verdadero ;** | | |
| 0 | *decl_atr* | *decl_atr* → **id :** *tipo* **=** *expr* **;** |
| 1 | **id :** *tipo* **=** *expr* **;** | *tipo* → **logico** |
| 2 | **id : logico =** *expr* **;** | *expr* ⇒* **verdadero** (G7) |
| 3 | **id : logico = verdadero ;** | *fin* |
| **Ejemplo 2.4**: gravedad: real = -9.8; | | |
| *Cadena de tokens:* **id : real = - dec ;** | | |
| 0 | *decl_atr* | *decl_atr* → **id :** *tipo* **=** *expr* **;** |
| 1 | **id :** *tipo* **=** *expr* **;** | *tipo* → **real** |
| 2 | **id : real =** *expr* **;** | *expr* ⇒* **- dec** (G7) |
| 3 | **id : real = - dec ;** | *fin* |

**Tabla B3**

*Derivaciones más a la izquierda de la gramática G3 (declaración de eventos)*

| Paso | Forma sentencial | Producción aplicada |
|---|---|---|
| **Ejemplo 3.1**: eventos saltar; | | |
| *Cadena de tokens:* **eventos id ;** | | |
| 0 | *decl_evs* | *decl_evs* → **eventos** *lista_ids* **;** |
| 1 | **eventos** *lista_ids* **;** | *lista_ids* → **id** |
| 2 | **eventos id ;** | *fin* |
| **Ejemplo 3.2**: eventos saltar, atacar; | | |
| *Cadena de tokens:* **eventos id , id ;** | | |
| 0 | *decl_evs* | *decl_evs* → **eventos** *lista_ids* **;** |
| 1 | **eventos** *lista_ids* **;** | *lista_ids* → *lista_ids* **, id** |
| 2 | **eventos** *lista_ids* **, id ;** | *lista_ids* → **id** |
| 3 | **eventos id , id ;** | *fin* |
| **Ejemplo 3.3**: eventos saltar, atacar, recibir_golpe; | | |
| *Cadena de tokens:* **eventos id , id , id ;** | | |
| 0 | *decl_evs* | *decl_evs* → **eventos** *lista_ids* **;** |
| 1 | **eventos** *lista_ids* **;** | *lista_ids* → *lista_ids* **, id** |

| Paso | Forma sentencial | Producción aplicada |
|---|---|---|
| 2 | **eventos** *lista_ids* **, id ;** | *lista_ids* → *lista_ids* **, id** |
| 3 | **eventos** *lista_ids* **, id , id ;** | *lista_ids* → **id** |
| 4 | **eventos id , id , id ;** | *fin* |
| **Ejemplo 3.5**: eventos pulsar_a, pulsar_b, soltar_a, soltar_b, pausar; | | |
| *Cadena de tokens:* **eventos id , id , id , id , id ;** | | |
| 0 | *decl_evs* | *decl_evs* → **eventos** *lista_ids* **;** |
| 1 | **eventos** *lista_ids* **;** | *lista_ids* → *lista_ids* **, id** |
| 2 | **eventos** *lista_ids* **, id ;** | *lista_ids* → *lista_ids* **, id** |
| 3 | **eventos** *lista_ids* **, id , id ;** | *lista_ids* → *lista_ids* **, id** |
| 4 | **eventos** *lista_ids* **, id , id , id ;** | *lista_ids* → *lista_ids* **, id** |
| 5 | **eventos** *lista_ids* **, id , id , id , id ;** | *lista_ids* → **id** |
| 6 | **eventos id , id , id , id , id ;** | *fin* |

**Tabla B4**

*Derivaciones más a la izquierda de la gramática G4 (declaración de estados)*

| Paso | Forma sentencial | Producción aplicada |
|---|---|---|
| **Ejemplo 4.1**: estado Reposo { } | | |
| *Cadena de tokens:* **estado id { }** | | |
| 0 | *decl_est* | *decl_est* → *modif* **estado id {** *entrada lista_trs* **}** |
| 1 | *modif* **estado id {** *entrada lista_trs* **}** | *modif* → ε |
| 2 | **estado id {** *entrada lista_trs* **}** | *entrada* → ε |
| 3 | **estado id {** *lista_trs* **}** | *lista_trs* → ε |
| 4 | **estado id { }** | *fin* |
| **Ejemplo 4.2**: inicial estado Reposo { cuando saltar -> Saltando; } | | |
| *Cadena de tokens:* **inicial estado id { cuando id -> id ; }** | | |
| 0 | *decl_est* | *decl_est* → *modif* **estado id {** *entrada lista_trs* **}** |
| 1 | *modif* **estado id {** *entrada lista_trs* **}** | *modif* → **inicial** |
| 2 | **inicial estado id {** *entrada lista_trs* **}** | *entrada* → ε |
| 3 | **inicial estado id {** *lista_trs* **}** | *lista_trs* → *lista_trs transicion* |
| 4 | **inicial estado id {** *lista_trs transicion* **}** | *lista_trs* → ε |
| 5 | **inicial estado id {** *transicion* **}** | *transicion* ⇒* **cuando id -> id ;** (G5) |
| 6 | **inicial estado id { cuando id -> id ; }** | *fin* |
| **Ejemplo 4.3**: final estado Derrotado { } | | |
| *Cadena de tokens:* **final estado id { }** | | |
| 0 | *decl_est* | *decl_est* → *modif* **estado id {** *entrada lista_trs* **}** |
| 1 | *modif* **estado id {** *entrada lista_trs* **}** | *modif* → **final** |
| 2 | **final estado id {** *entrada lista_trs* **}** | *entrada* → ε |

| Paso | Forma sentencial | Producción aplicada |
|---|---|---|
| 3 | **final estado id {** *lista_trs* **}** | *lista_trs* → ε |
| 4 | **final estado id { }** | *fin* |
| **Ejemplo 4.4** (código en el Apéndice A) | | |
| *Cadena de tokens:* **estado id { al_entrar { id = id - id ; } cuando id -> id ; }** | | |
| 0 | *decl_est* | *decl_est* → *modif* **estado id {** *entrada lista_trs* **}** |
| 1 | *modif* **estado id {** *entrada lista_trs* **}** | *modif* → ε |
| 2 | **estado id {** *entrada lista_trs* **}** | *entrada* → **al_entrar** *bloque* |
| 3 | **estado id { al_entrar** *bloque lista_trs* **}** | *bloque* ⇒* **{ id = id - id ; }** (G6) |
| 4 | **estado id { al_entrar { id = id - id ; }** *lista_trs* **}** | *lista_trs* → *lista_trs transicion* |
| 5 | **estado id { al_entrar { id = id - id ; }** *lista_trs transicion* **}** | *lista_trs* → ε |
| 6 | **estado id { al_entrar { id = id - id ; }** *transicion* **}** | *transicion* ⇒* **cuando id -> id ;** (G5) |
| 7 | **estado id { al_entrar { id = id - id ; } cuando id -> id ; }** | *fin* |

**Tabla B5**

*Derivaciones más a la izquierda de la gramática G5 (transiciones)*

| Paso | Forma sentencial | Producción aplicada |
|---|---|---|
| **Ejemplo 5.1**: cuando saltar -> Saltando; | | |
| *Cadena de tokens:* **cuando id -> id ;** | | |
| 0 | *transicion* | *transicion* → **cuando id** *guarda* **-> id ;** |
| 1 | **cuando id** *guarda* **-> id ;** | *guarda* → ε |
| 2 | **cuando id -> id ;** | *fin* |
| **Ejemplo 5.2**: cuando fin_aturdimiento si vida <= 0 -> Derrotado; | | |
| *Cadena de tokens:* **cuando id si id <= num -> id ;** | | |
| 0 | *transicion* | *transicion* → **cuando id** *guarda* **-> id ;** |
| 1 | **cuando id** *guarda* **-> id ;** | *guarda* → **si** *expr* |
| 2 | **cuando id si** *expr* **-> id ;** | *expr* ⇒* **id <= num** (G7) |
| 3 | **cuando id si id <= num -> id ;** | *fin* |
| **Ejemplo 5.4**: cuando disparar si municion > 0 && !bloqueada -> Disparando; | | |
| *Cadena de tokens:* **cuando id si id > num && ! id -> id ;** | | |
| 0 | *transicion* | *transicion* → **cuando id** *guarda* **-> id ;** |
| 1 | **cuando id** *guarda* **-> id ;** | *guarda* → **si** *expr* |
| 2 | **cuando id si** *expr* **-> id ;** | *expr* ⇒* **id > num && ! id** (G7) |
| 3 | **cuando id si id > num && ! id -> id ;** | *fin* |
| **Ejemplo 5.5**: cuando recibir_golpe si escudo \|\| vida > 50 -> Bloqueando; | | |
| *Cadena de tokens:* **cuando id si id \|\| id > num -> id ;** | | |

| Paso | Forma sentencial | Producción aplicada |
|---|---|---|
| 0 | *transicion* | *transicion* → **cuando id** *guarda* **-> id ;** |
| 1 | **cuando id** *guarda* **-> id ;** | *guarda* → **si** *expr* |
| 2 | **cuando id si** *expr* **-> id ;** | *expr* ⇒* **id \|\| id > num** (G7) |
| 3 | **cuando id si id \|\| id > num -> id ;** | *fin* |

**Tabla B6**

*Derivaciones más a la izquierda de la gramática G6 (acciones: asignación y selección)*

| Paso | Forma sentencial | Producción aplicada |
|---|---|---|
| **Ejemplo 6.1**: vida = 100; | | |
| *Cadena de tokens:* **id = num ;** | | |
| 0 | *sentencia* | *sentencia* → **id =** *expr* **;** |
| 1 | **id =** *expr* **;** | *expr* ⇒* **num** (G7) |
| 2 | **id = num ;** | *fin* |
| **Ejemplo 6.2**: velocidad = velocidad * 0.5; | | |
| *Cadena de tokens:* **id = id * dec ;** | | |
| 0 | *sentencia* | *sentencia* → **id =** *expr* **;** |
| 1 | **id =** *expr* **;** | *expr* ⇒* **id * dec** (G7) |
| 2 | **id = id * dec ;** | *fin* |
| **Ejemplo 6.3**: si vida < 50 { velocidad = 1.0; } | | |
| *Cadena de tokens:* **si id < num { id = dec ; }** | | |
| 0 | *sentencia* | *sentencia* → **si** *expr bloque alternativa* |
| 1 | **si** *expr bloque alternativa* | *expr* ⇒* **id < num** (G7) |
| 2 | **si id < num** *bloque alternativa* | *bloque* → **{** *lista_sent* **}** |
| 3 | **si id < num {** *lista_sent* **}** *alternativa* | *lista_sent* → *lista_sent sentencia* |
| 4 | **si id < num {** *lista_sent sentencia* **}** *alternativa* | *lista_sent* → ε |
| 5 | **si id < num {** *sentencia* **}** *alternativa* | *sentencia* → **id =** *expr* **;** |
| 6 | **si id < num { id =** *expr* **; }** *alternativa* | *expr* ⇒* **dec** (G7) |
| 7 | **si id < num { id = dec ; }** *alternativa* | *alternativa* → ε |
| 8 | **si id < num { id = dec ; }** | *fin* |
| **Ejemplo 6.4**: si vida < 50 { velocidad = 1.0; } sino { velocidad = 1.5; } | | |
| *Cadena de tokens:* **si id < num { id = dec ; } sino { id = dec ; }** | | |
| 0 | *sentencia* | *sentencia* → **si** *expr bloque alternativa* |
| 1 | **si** *expr bloque alternativa* | *expr* ⇒* **id < num** (G7) |
| 2 | **si id < num** *bloque alternativa* | *bloque* → **{** *lista_sent* **}** |
| 3 | **si id < num {** *lista_sent* **}** *alternativa* | *lista_sent* → *lista_sent sentencia* |
| 4 | **si id < num {** *lista_sent sentencia* **}** *alternativa* | *lista_sent* → ε |
| 5 | **si id < num {** *sentencia* **}** *alternativa* | *sentencia* → **id =** *expr* **;** |
| 6 | **si id < num { id =** *expr* **; }** *alternativa* | *expr* ⇒* **dec** (G7) |

| Paso | Forma sentencial | Producción aplicada |
|---|---|---|
| 7 | **si id < num { id = dec ; }** *alternativa* | *alternativa* → **sino** *bloque* |
| 8 | **si id < num { id = dec ; } sino** *bloque* | *bloque* → **{** *lista_sent* **}** |
| 9 | **si id < num { id = dec ; } sino {** *lista_sent* **}** | *lista_sent* → *lista_sent sentencia* |
| 10 | **si id < num { id = dec ; } sino {** *lista_sent sentencia* **}** | *lista_sent* → ε |
| 11 | **si id < num { id = dec ; } sino {** *sentencia* **}** | *sentencia* → **id =** *expr* **;** |
| 12 | **si id < num { id = dec ; } sino { id =** *expr* **; }** | *expr* ⇒* **dec** (G7) |
| 13 | **si id < num { id = dec ; } sino { id = dec ; }** | *fin* |

**Tabla B7**

*Derivaciones más a la izquierda de la gramática G7 (expresiones)*

| Paso | Forma sentencial | Producción aplicada |
|---|---|---|
| **Ejemplo 7.1**: vida - 25 < 10 | | |
| *Cadena de tokens:* **id - num < num** | | |
| 0 | *expr* | *expr* → *conj* |
| 1 | *conj* | *conj* → *rel* |
| 2 | *rel* | *rel* → *arit oprel arit* |
| 3 | *arit oprel arit* | *arit* → *arit* **-** *term* |
| 4 | *arit* **-** *term oprel arit* | *arit* → *term* |
| 5 | *term* **-** *term oprel arit* | *term* → *unario* |
| 6 | *unario* **-** *term oprel arit* | *unario* → *factor* |
| 7 | *factor* **-** *term oprel arit* | *factor* → **id** |
| 8 | **id -** *term oprel arit* | *term* → *unario* |
| 9 | **id -** *unario oprel arit* | *unario* → *factor* |
| 10 | **id -** *factor oprel arit* | *factor* → **num** |
| 11 | **id - num** *oprel arit* | *oprel* → **<** |
| 12 | **id - num <** *arit* | *arit* → *term* |
| 13 | **id - num <** *term* | *term* → *unario* |
| 14 | **id - num <** *unario* | *unario* → *factor* |
| 15 | **id - num <** *factor* | *factor* → **num** |
| 16 | **id - num < num** | *fin* |
| **Ejemplo 7.2**: 2 * (velocidad + 1.5) / 4 | | |
| *Cadena de tokens:* **num * ( id + dec ) / num** | | |
| 0 | *expr* | *expr* → *conj* |
| 1 | *conj* | *conj* → *rel* |
| 2 | *rel* | *rel* → *arit* |
| 3 | *arit* | *arit* → *term* |
| 4 | *term* | *term* → *term* **/** *unario* |
| 5 | *term* **/** *unario* | *term* → *term* ***** *unario* |
| 6 | *term* ***** *unario* **/** *unario* | *term* → *unario* |
| 7 | *unario* ***** *unario* **/** *unario* | *unario* → *factor* |
| 8 | *factor* ***** *unario* **/** *unario* | *factor* → **num** |

| Paso | Forma sentencial | Producción aplicada |
|---|---|---|
| 9 | **num *** *unario* / *unario* | *unario* → *factor* |
| 10 | **num *** *factor* / *unario* | *factor* → **(** *expr* **)** |
| 11 | **num * (** *expr* **)** / *unario* | *expr* → *conj* |
| 12 | **num * (** *conj* **)** / *unario* | *conj* → *rel* |
| 13 | **num * (** *rel* **)** / *unario* | *rel* → *arit* |
| 14 | **num * (** *arit* **)** / *unario* | *arit* → *arit* **+** *term* |
| 15 | **num * (** *arit* **+** *term* **)** / *unario* | *arit* → *term* |
| 16 | **num * (** *term* **+** *term* **)** / *unario* | *term* → *unario* |
| 17 | **num * (** *unario* **+** *term* **)** / *unario* | *unario* → *factor* |
| 18 | **num * (** *factor* **+** *term* **)** / *unario* | *factor* → **id** |
| 19 | **num * ( id +** *term* **)** / *unario* | *term* → *unario* |
| 20 | **num * ( id +** *unario* **)** / *unario* | *unario* → *factor* |
| 21 | **num * ( id +** *factor* **)** / *unario* | *factor* → **dec** |
| 22 | **num * ( id + dec )** / *unario* | *unario* → *factor* |
| 23 | **num * ( id + dec )** / *factor* | *factor* → **num** |
| 24 | **num * ( id + dec ) / num** | *fin* |
| **Ejemplo 7.3**: vida <= 0 \|\| !vivo | | |
| *Cadena de tokens:* **id <= num \|\| ! id** | | |
| 0 | *expr* | *expr* → *expr* **||** *conj* |
| 1 | *expr* **||** *conj* | *expr* → *conj* |
| 2 | *conj* **||** *conj* | *conj* → *rel* |
| 3 | *rel* **||** *conj* | *rel* → *arit oprel arit* |
| 4 | *arit oprel arit* **||** *conj* | *arit* → *term* |
| 5 | *term oprel arit* **||** *conj* | *term* → *unario* |
| 6 | *unario oprel arit* **||** *conj* | *unario* → *factor* |
| 7 | *factor oprel arit* **||** *conj* | *factor* → **id** |
| 8 | **id** *oprel arit* **||** *conj* | *oprel* → **<=** |
| 9 | **id <=** *arit* **||** *conj* | *arit* → *term* |
| 10 | **id <=** *term* **||** *conj* | *term* → *unario* |
| 11 | **id <=** *unario* **||** *conj* | *unario* → *factor* |
| 12 | **id <=** *factor* **||** *conj* | *factor* → **num** |
| 13 | **id <= num ||** *conj* | *conj* → *rel* |
| 14 | **id <= num ||** *rel* | *rel* → *arit* |
| 15 | **id <= num ||** *arit* | *arit* → *term* |
| 16 | **id <= num ||** *term* | *term* → *unario* |
| 17 | **id <= num ||** *unario* | *unario* → **!** *unario* |
| 18 | **id <= num || !** *unario* | *unario* → *factor* |
| 19 | **id <= num || !** *factor* | *factor* → **id** |
| 20 | **id <= num || ! id** | *fin* |
| **Ejemplo 7.4**: municion > 0 && escudo != verdadero | | |
| *Cadena de tokens:* **id > num && id != verdadero** | | |

| Paso | Forma sentencial | Producción aplicada |
|---|---|---|
| 0 | *expr* | *expr* → *conj* |
| 1 | *conj* | *conj* → *conj* **&&** *rel* |
| 2 | *conj* **&&** *rel* | *conj* → *rel* |
| 3 | *rel* **&&** *rel* | *rel* → *arit oprel arit* |
| 4 | *arit oprel arit* **&&** *rel* | *arit* → *term* |
| 5 | *term oprel arit* **&&** *rel* | *term* → *unario* |
| 6 | *unario oprel arit* **&&** *rel* | *unario* → *factor* |
| 7 | *factor oprel arit* **&&** *rel* | *factor* → **id** |
| 8 | **id** *oprel arit* **&&** *rel* | *oprel* → **>** |
| 9 | **id** **>** *arit* **&&** *rel* | *arit* → *term* |
| 10 | **id** **>** *term* **&&** *rel* | *term* → *unario* |
| 11 | **id** **>** *unario* **&&** *rel* | *unario* → *factor* |
| 12 | **id** **>** *factor* **&&** *rel* | *factor* → **num** |
| 13 | **id** **>** **num** **&&** *rel* | *rel* → *arit oprel arit* |
| 14 | **id** **>** **num** **&&** *arit oprel arit* | *arit* → *term* |
| 15 | **id** **>** **num** **&&** *term oprel arit* | *term* → *unario* |
| 16 | **id** **>** **num** **&&** *unario oprel arit* | *unario* → *factor* |
| 17 | **id** **>** **num** **&&** *factor oprel arit* | *factor* → **id** |
| 18 | **id** **>** **num** **&&** **id** *oprel arit* | *oprel* → **!=** |
| 19 | **id** **>** **num** **&&** **id** **!=** *arit* | *arit* → *term* |
| 20 | **id** **>** **num** **&&** **id** **!=** *term* | *term* → *unario* |
| 21 | **id** **>** **num** **&&** **id** **!=** *unario* | *unario* → *factor* |
| 22 | **id** **>** **num** **&&** **id** **!=** *factor* | *factor* → **verdadero** |
| 23 | **id** **>** **num** **&&** **id** **!=** **verdadero** | *fin* |

# Apéndice C
# Pruebas de los analizadores

Las pruebas están en la carpeta *pruebas* del repositorio del proyecto (https://github.com/DavidRamPari/pixellang). Las entradas léxicas son fragmentos de programa, por lo que se analizan solo con el analizador léxico, mediante pruebas/correr_pruebas.py; las sintácticas y las semánticas se ejecutan con el driver. Cada tabla muestra la entrada y la salida obtenida.

## Analizador léxico

La Tabla C1 muestra los tokens que se obtienen de una entrada válida; el comentario no produce ninguno. La Tabla C2 muestra entradas con caracteres o lexemas que no pertenecen al léxico.

**Tabla C1**
*Tokens de una entrada válida*

| Línea | Entrada y tokens |
|---|---|
| 1 | daño_golpe: entero = 25;<br>*ID DOS_PUNTOS ENTERO ASIGNA NUM_ENTERO PUNTO_COMA* |
| 2 | vivo: lógico = verdadero;<br>*ID DOS_PUNTOS LOGICO ASIGNA VERDADERO PUNTO_COMA* |
| 3 | cuando fin_aturdimiento si vida <= 0 -> Derrotado; // comentario<br>*CUANDO ID SI ID MENOR_IGUAL NUM_ENTERO FLECHA ID PUNTO_COMA* |

*Nota.* En cada fila, la primera línea es la entrada y la segunda, en cursiva, la lista de tokens.

**Tabla C2**
*Pruebas del analizador léxico con entradas inválidas*

| Entrada | Salida del analizador léxico |
|---|---|
| **L1.** El carácter @ no pertenece a ningún patrón. | |
| vida: entero = 100 @ 2; | Error léxico en la línea 1, columna 19:<br>token recognition error at: '@' |
| **L2.** Falta un dígito después del punto: 2 es NUM_ENTERO y el punto queda solo. | |
| velocidad: real = 2.; | Error léxico en la línea 1, columna 19:<br>token recognition error at: '.' |
| **L3.** El carácter # no pertenece al léxico; atacar se reconoce como ID. | |
| eventos saltar, #atacar; | Error léxico en la línea 1, columna 16:<br>token recognition error at: '#' |
| **L4.** 3.14 es NUM_REAL y el segundo punto queda sin regla. | |
| y: real = 3.14.15; | Error léxico en la línea 1, columna 14:<br>token recognition error at: '.' |
| **L5.** El analizador léxico intenta reconocer && y falla en el espacio. | |
| vivo: logico = verdadero & falso; | Error léxico en la línea 1, columna 25:<br>token recognition error at: '& ' |

## Analizador sintáctico

Los 35 ejemplos del Apéndice A se analizan sin errores. La Tabla C3 muestra programas con errores sintácticos; los mensajes son los de ANTLR.

**Tabla C3**
*Pruebas del analizador sintáctico*

| **Fragmento del programa** | **Salida del driver** |
|---|---|
| **S1.** Falta el punto y coma al final de la transición. | |
| inicial estado Reposo {<br>  cuando recibir_golpe -> Aturdido<br>} | Error sintáctico en la línea 5, columna 4:<br>  missing ';' at '}' |
| **S2.** Falta la flecha -> entre el evento y el destino. | |
| inicial estado Reposo {<br>  cuando recibir_golpe Aturdido;<br>} | Error sintáctico en la línea 4, columna 29:<br>  mismatched input 'Aturdido' expecting {'si', '->'} |
| **S3.** La selección si está escrita sin llaves. | |
| al_entrar { si vida < 50 velocidad = 0.5; }<br>  cuando recibir_golpe -> Reposo; | Error sintáctico en la línea 6, columna 33:<br>  missing '{' at 'velocidad'<br>Error sintáctico en la línea 7, columna 8:<br>  extraneous input 'cuando' expecting {'si', '}', ID} |
| **S4.** Falta la declaración de eventos. | |
| personaje Caballero {<br>  vida: entero = 100;<br>  inicial estado Reposo { }<br>} | Error sintáctico en la línea 3, columna 4:<br>  extraneous input 'inicial' expecting {'eventos', ID}<br>Error sintáctico en la línea 3, columna 26:<br>  mismatched input '{' expecting ':' |
| **S5.** Hay una transición fuera de un estado. | |
| eventos recibir_golpe;<br>cuando recibir_golpe -> Aturdido;<br>inicial estado Reposo { } | Error sintáctico en la línea 3, columna 4:<br>  mismatched input 'cuando' expecting {'estado', 'inicial', 'final'} |
| **S6.** El estado no tiene nombre. | |
| inicial estado { cuando recibir_golpe -> Reposo; } | Error sintáctico en la línea 3, columna 19:<br>  missing ID at '{' |

## Analizador semántico

Cada prueba de la Tabla C4 es un programa correcto en lo léxico y lo sintáctico que contiene uno de los diez errores semánticos. Los programas ejemplos/caballero.pxl y ejemplos/slime.pxl, en cambio, no tienen errores, y el driver termina con “Sin errores léxicos, sintácticos ni semánticos.”

**Tabla C4**

*Pruebas del analizador semántico*

| Fragmento del programa | Salida del driver |
|---|---|
| **E01.** La transición apunta a Volando, que no se declaró. | |
| estado Saltando {<br>  cuando aterrizar -> Reposo;<br>  cuando saltar -> Volando;<br>} | Error semántico E01 en la línea 6, columna 25: la transición de 'Saltando' con 'saltar' va hacia 'Volando', que no es un estado declarado |
| **E02.** Los eventos caer y agacharse no están declarados. | |
| estado Saltando { cuando caer -> Reposo; }<br>simular Caballero: saltar, aterrizar, agacharse; | Error semántico E02 en la línea 4, columna 29: el evento 'caer' no está declarado en 'Caballero'<br>Error semántico E02 en la línea 6, columna 38: el evento 'agacharse' no está declarado en 'Caballero' |
| **E03.** La transición sin guarda está antes que la que tiene guarda. | |
| // estado Aturdido<br>cuando recibir_golpe -> Aturdido;<br>cuando fin_aturdimiento -> Reposo;<br>cuando fin_aturdimiento si vida <= 0 -><br>  Derrotado; | Error semántico E03 en la línea 9, columna 8: en 'Aturdido', la transición con 'fin_aturdimiento' de la línea 9 nunca se toma: la de la línea 8 responde al mismo evento sin guarda |
| **E04.** El personaje Caballero no tiene estado inicial y Slime tiene dos. | |
| // personaje Caballero<br>estado Reposo { cuando saltar -> Saltando; }<br>estado Saltando { cuando aterrizar -> Reposo; }<br>// personaje Slime<br>inicial estado Patrulla {<br>inicial estado Aplastado { | Error semántico E04 en la línea 1, columna 10: el personaje 'Caballero' no tiene estado inicial<br>Error semántico E04 en la línea 11, columna 19: el personaje 'Slime' tiene más de un estado inicial: 'Patrulla', 'Aplastado' |
| **E05.** Ninguna transición lleva a Nadando. | |
| // Reposo: cuando saltar -> Saltando;<br>estado Saltando { cuando aterrizar -> Reposo; }<br>estado Nadando { cuando aterrizar -> Reposo; } | Error semántico E05 en la línea 5, columna 11: el estado 'Nadando' no es alcanzable desde 'Reposo' |
| **E06.** Se repiten un atributo y un evento, y un estado usa el nombre de un evento. | |
| vida: entero = 100;<br>vida: real = 50.0;<br>eventos saltar, recibir_golpe, recibir_golpe;<br>inicial estado Reposo { cuando saltar -> saltar; }<br>estado saltar { cuando recibir_golpe -><br>  Reposo; } | Error semántico E06 en la línea 3, columna 4: el atributo 'vida' ya fue declarado en la línea 2<br>Error semántico E06 en la línea 4, columna 35: el evento 'recibir_golpe' ya fue declarado en la línea 4<br>Error semántico E06 en la línea 6, columna 11: 'saltar' ya es el nombre de un evento (línea 4); un nombre no puede usarse para dos cosas distintas |

| Fragmento del programa | Salida del driver |
|---|---|
| **E07.** Se usan escudo y energia, que no son atributos del personaje. | |
| al_entrar { escudo = 10; }<br>cuando recibir_golpe si energia > 0 -> Reposo; | Error semántico E07 en la línea 5, columna 20: 'escudo' no es un atributo de 'Caballero'<br>Error semántico E07 en la línea 6, columna 32: 'energia' no es un atributo de 'Caballero' |
| **E08.** Tipos que no corresponden en una inicialización, una suma, una condición y una guarda. | |
| vida: entero = 2.5;<br>vida = vivo + 1;<br>si vida { vivo = falso; }<br>cuando recibir_golpe si vida -> Reposo; | Error semántico E08 en la línea 2, columna 4: el atributo 'vida' es entero y se inicializa con un valor real<br>Error semántico E08 en la línea 7, columna 19: el operador '+' necesita números y recibió logico<br>Error semántico E08 en la línea 8, columna 15: la condición de 'si' debe ser logico, no entero<br>Error semántico E08 en la línea 10, columna 32: la guarda de la transición debe ser logico, no entero |
| **E09.** Derrotado es final, pero tiene una transición de salida. | |
| final estado Derrotado {<br>cuando revivir -> Reposo;<br>} | Error semántico E09 en la línea 6, columna 17: 'Derrotado' es un estado final y no puede tener transiciones |
| **E10.** La simulación se refiere a Mago, que no existe. | |
| simular Mago: recibir_golpe; | Error semántico E10 en la línea 6, columna 8: no existe un personaje llamado 'Mago' |

# Apéndice D
# Código intermedio y pruebas del hito 2

Este apéndice muestra el código intermedio que produce el compilador para ejemplos/caballero.pxl, la simulación de ese código y la salida de las pruebas automáticas del hito 2. La Tabla 7 indica la opción del driver que muestra cada resultado, y todo se puede reproducir con los comandos de la guía del repositorio (GUIA.md).

## Código de tres direcciones

La Tabla D1 contiene los 13 procedimientos del caballero. La columna izquierda tiene el procedimiento que lo inicia y los que entran a cada estado; la derecha, los que procesan cada evento y el de la simulación.

**Tabla D1**
*Código de tres direcciones de ejemplos/caballero.pxl*

| **Iniciar y entrar a cada estado** | **Procesar cada evento y simular** |
|---|---|
| Caballero.iniciar:<br>  vida = 100<br>  daño_golpe = 25<br>  velocidad = 0.0<br>  y = 0.0<br>  call Caballero.al_entrar_Reposo, 0<br>  return | Caballero.cuando_saltar:<br>  t4 = estado == Reposo<br>  ifFalse t4 goto L4<br>  call Caballero.al_entrar_Saltando, 0<br>  return<br>L4:<br>  return |
| Caballero.al_entrar_Reposo:<br>  estado = Reposo<br>  velocidad = 0.0<br>  y = 0.0<br>  return | Caballero.cuando_atacar:<br>  t5 = estado == Reposo<br>  ifFalse t5 goto L5<br>  call Caballero.al_entrar_Atacando, 0<br>  return<br>L5:<br>  return |
| Caballero.al_entrar_Saltando:<br>  estado = Saltando<br>  t1 = y + 2.5<br>  y = t1<br>  return | Caballero.cuando_aterrizar:<br>  t6 = estado == Saltando<br>  ifFalse t6 goto L6<br>  call Caballero.al_entrar_Reposo, 0<br>  return<br>L6:<br>  return |
| Caballero.al_entrar_Atacando:<br>  estado = Atacando<br>  t2 = vida < 50<br>  if t2 goto L1<br>  goto L2<br>L1:<br>  velocidad = 1.0<br>  goto L3<br>L2:<br>  velocidad = 1.5<br>L3:<br>  return | Caballero.cuando_fin_ataque:<br>  t7 = estado == Atacando<br>  ifFalse t7 goto L7<br>  call Caballero.al_entrar_Reposo, 0<br>  return<br>L7:<br>  return |

| Iniciar y entrar a cada estado | Procesar cada evento y simular |
|---|---|
| Caballero.al_entrar_Aturdido:<br>estado = Aturdido<br>t3 = vida - daño_golpe<br>vida = t3<br>velocidad = 0.0<br>return | Caballero.cuando_recibir_golpe:<br>t8 = estado == Reposo<br>ifFalse t8 goto L8<br>call Caballero.al_entrar_Aturdido, 0<br>return<br>L8:<br>t9 = estado == Saltando<br>ifFalse t9 goto L9<br>call Caballero.al_entrar_Aturdido, 0<br>return<br>L9:<br>t10 = estado == Atacando<br>ifFalse t10 goto L10<br>call Caballero.al_entrar_Aturdido, 0<br>return<br>L10:<br>t11 = estado == Aturdido<br>ifFalse t11 goto L11<br>call Caballero.al_entrar_Aturdido, 0<br>return<br>L11:<br>return |
| Caballero.al_entrar_Derrotado:<br>estado = Derrotado<br>vida = 0<br>return | Caballero.cuando_fin_aturdimiento:<br>t12 = estado == Aturdido<br>ifFalse t12 goto L12<br>t13 = vida <= 0<br>if t13 goto L13<br>goto L14<br>L13:<br>call Caballero.al_entrar_Derrotado, 0<br>return<br>L14:<br>call Caballero.al_entrar_Reposo, 0<br>return<br>L12:<br>return |
| | simular_1:<br>call Caballero.iniciar, 0<br>call Caballero.cuando_saltar, 0<br>call Caballero.cuando_aterrizar, 0<br>call Caballero.cuando_atacar, 0<br>call Caballero.cuando_fin_ataque, 0<br>call Caballero.cuando_recibir_golpe, 0<br>call Caballero.cuando_recibir_golpe, 0<br>call Caballero.cuando_recibir_golpe, 0<br>call Caballero.cuando_fin_aturdimiento, 0<br>call Caballero.cuando_atacar, 0<br>call Caballero.cuando_fin_ataque, 0<br>call Caballero.cuando_recibir_golpe, 0<br>call Caballero.cuando_fin_aturdimiento, 0<br>return |

## Cuádruplos y triples

La Tabla D2 muestra el procedimiento Caballero.al_entrar_Atacando como cuádruplos y como triples. La posición es la de la instrucción en todo el código del programa; en los triples, el temporal t2 se reemplaza por (19), la posición de la instrucción que lo calcula.

**Tabla D2**

*Cuádruplos y triples del procedimiento Caballero.al_entrar_Atacando*

| Posición | op | arg1 | arg2 | resultado | Triple |
|---|---|---|---|---|---|
| 17 | label | | | Caballero. al_entrar_Atacando | (label, -, Caballero. al_entrar_Atacando) |
| 18 | copy | Atacando | | estado | (copy, Atacando, estado) |
| 19 | < | vida | 50 | t2 | (<, vida, 50) |
| 20 | if | t2 | | L1 | (if, (19), L1) |
| 21 | goto | | | L2 | (goto, -, L2) |
| 22 | label | | | L1 | (label, -, L1) |
| 23 | copy | 1.0 | | velocidad | (copy, 1.0, velocidad) |
| 24 | goto | | | L3 | (goto, -, L3) |
| 25 | label | | | L2 | (label, -, L2) |
| 26 | copy | 1.5 | | velocidad | (copy, 1.5, velocidad) |
| 27 | label | | | L3 | (label, -, L3) |
| 28 | return | | | | (return, -, -) |

## Simulación sobre el código intermedio

La Tabla D3 muestra lo que produce la máquina de TAC al ejecutar el bloque simular del caballero: después de cada evento, el estado y el valor de cada atributo.

**Tabla D3**

*Simulación del caballero sobre el código de tres direcciones*

| Evento | Estado | vida | daño_golpe | velocidad | y |
|---|---|---|---|---|---|
| inicio | Reposo | 100 | 25 | 0.0 | 0.0 |
| saltar | Saltando | 100 | 25 | 0.0 | 2.5 |
| aterrizar | Reposo | 100 | 25 | 0.0 | 0.0 |
| atacar | Atacando | 100 | 25 | 1.5 | 0.0 |
| fin_ataque | Reposo | 100 | 25 | 0.0 | 0.0 |
| recibir_golpe | Aturdido | 75 | 25 | 0.0 | 0.0 |
| recibir_golpe | Aturdido | 50 | 25 | 0.0 | 0.0 |
| recibir_golpe | Aturdido | 25 | 25 | 0.0 | 0.0 |

| Evento | Estado | vida | daño_golpe | velocidad | y |
|---|---|---|---|---|---|
| fin_aturdimiento | Reposo | 25 | 25 | 0.0 | 0.0 |
| atacar | Atacando | 25 | 25 | 1.0 | 0.0 |
| fin_ataque | Reposo | 25 | 25 | 0.0 | 0.0 |
| recibir_golpe | Aturdido | 0 | 25 | 0.0 | 0.0 |
| fin_aturdimiento | Derrotado | 0 | 25 | 0.0 | 0.0 |

## Pruebas automáticas

La Tabla D4 muestra la salida de las tres pruebas nuevas del hito 2. La prueba diferencial se ejecutó con 3000 programas y la semilla 7.

**Tabla D4**

*Salida de las pruebas del hito 2*

| Comando | Salida |
|---|---|
| python pruebas/probar_ast.py | Expresiones: OK<br>Programa completo: OK<br>Listado del AST: OK |
| python pruebas/probar_tac.py | Expresiones, cuádruplos y triples: OK<br>Conversiones y operadores unarios: OK<br>Cortocircuito y si/sino: OK<br>Nombres de temporales: OK<br>Simulación: OK |
| python pruebas/probar_generados.py 3000 7 | 3000 programas generados, 6779 simulaciones y 40210 eventos comparados (1842 simulaciones terminan en un error de ejecución)<br>fallos: 0 |